

Project 3 Test Data

Step 6 for Project 3 is split into two parts. The first part is finding the optima of 2 feature selection landscapes. And the second part is explaining why your algorithm(s) fails on a difficult synthetic problem.

In both instances you're supposed to run on the test instances as if you didn't know what the landscapes looked like, in a live-run "approximation", because in most applications in industry you wouldn't know what the landscape looks like (why use optimization algorithm if you have the global optima?). As such you are expected to keep the parameters the same as you did for the training datasets, although Parameter control is allowed if you implemented it earlier. You are also allowed to have scaling of parameters for example n^2 , as mentioned in the project description. You should use **random initialization**.

Task 1: Feature Selection

Provided in this folder are 2 datasets, they are of the same format as the feature selection training data. First we will explain what you should include in the report, then what the datasets are.

What to include in the Report for Feature Selection:

For this part of Project 3 you should include:

1. The individual x^* that achieved the solution, and its function value $f(x^*)$, aka it's fitness
2. A screenshot or visualization showing your algorithm reaching this position in the search space

Some could Includes:

3. If you did not reach the optima, provide the hamming distance to the optima, and a quick explanation of why you think it failed.
4. You can also provide visualizations and similar in the appendix.

You get 1 point if you are in the correct position, and diminishing points for every hamming distance you are away.

Datasets:

Use the Training accuracy as described in the project description as the fitness function (Eq. 1). For both Datasets the regularization is 0, that is to say there is no penalty. That is to say you ignore the part of the formula after the subtraction sign. The format of the data

is exactly the same as for the training data. We will publish the global optima on Friday (24/04/26), although finding them yourself should not be difficult, but we release it before just in case there is something wrong.

Dataset 1:

[Zoo](#)

Size/n: 16

Dataset 2:

[Hepatitis](#)

Size/n: 19

Task 2: Synthetic Landscape

This part of the the testing will build on the triangle function that was provided in the Project Description and the Project Lecture. This time however we're significantly scaling up the size of the dataset, and applying an extra rule. We expect your algorithm(s) to fail finding the global optima for this landscape, and that is in fact a part of the goal. You are expected to explain **WHY** your algorithm(s) fail to find the global optima and write it in your report and present it in the presentation. **Hint:** It is very likely you will be asked questions related to this as well.

What to include in the Report for Feature Selection:

You should provide the same information as what you provided for the Feature Selection landscapes, the visualizations of algorithm behaviour will be especially useful for you to understand what is happening.

We expect you to elaborate why these aspects makes finding the global optima for this problem difficult for this specific

- 1) The Fitness landscape and it's characteristics: This sounds vague, but let's give an **example:**

If you want to find a treasure using only a topographic map with an X that marks the treasure, it would be difficult in a massive place with no ruggedness and where everything looks the same, but the landscape would be easy to walk through because everything is relatively flat.

However a smaller area filled with peaks and valleys and tons of characteristic features would be much easier to find the treasure on, but harder to navigate.

- 2) Now you want to explain why your algorithm struggles on traversing the specified landscape. It might be easier to first think very generically what Evolutionary Algorithms do, and how they behave and evolve over time, then think why the

characteristics of the landscape would make it difficult to solve.

Then later you can connect it to specific parts of your algorithm, and think does this part affect the traversal and make it easier or more difficult for the algorithm to find the goal.

- 3) Then reflect briefly on what possible solutions could be implemented to help with this problem

Example of a single characteristic: I want to get to Himmelrike (the nature preserve north of the Trondheim Fjord). I know it's somewhere north, but it is dark so I can't really see well. I would take ages finding it because a part of my Algorithm behaviour is that I cannot walk on water, plus I don't want to swim. So I would have to walk continuously walk around the Fjord, which would take ages, and likely would take more steps than I would consider. So I don't make it to Himmelrike. So my Search Algorithm fails. A possible solution to this is having a boat, or finding another person that would swim across the Fjord to do the search, etc.

The examples provided above are somewhat silly, but they are hopefully understandable, please write something on blackboard if something is unclear.

The Landscape is defined as follows:

It is the triangle function, same as what you used in the training data, except slightly different. You can read about Asymmetric Triangle functions [here](#). Although I will just briefly explain what the difference is later. Please look at the explanation figure in the project description and lecture to relearn if you have forgotten:

When talking about bits below I mean how many active bits there are in the position in the fitness landscape. Same as it was for the training data

New Triangle function:

- $n=31$
- $s=5$
- $m=1$
- This time however there is an exception, from bits 26 and onwards s gets changed to 4
- Another exception is that from bits 30, m changes to 5.

So you end up with a list of fitnesses that look like this (**Updated**):

[0, 1, 2, 3, 4, 5, 4, 3, 2, 1, 0, 1, 2, 3, 4, 5, 4, 3, 2, 1, 0, 1, 2, 3, 4, 5, 4, 3, 2, 1, 0, 6]

Where the index is how many active bits there are for this particular bitstring (in that binary representation of the decimal value which is the index in the full fitness landscape).

There is no Penalty for this landscape either.

IF YOU SOLVE THE LANDSCAPE, WE STILL EXPECT YOU TO DEFINE WHY IT IS DIFFICULT, BUT WITH SOME INFORMATION OF WHY YOU SUCCEEDED.

You get max 1 point, if we think you provide sufficient understanding of how the landscape makes EA traversal tricky, and why your specific algorithm fails. We do not expect a massive amount of detail, but a slide in the presentation is good, and a paragraph or so in the Report of sufficient quality. Say you mention at least 2 aspects of the landscape's structure that makes it hard, and why EA's struggle with that, and reflect on 1-3 parts of your algorithm that make it harder or less hard to solve, It can be quite brief.

Important Tips:

The total size of the last landscape is 2 GiB (GibiByte). Every PC used for this course should have that available, however if you're using doubles or longs to represent the fitness it will take 16 GiB. So we highly recommend you change it to uint8's or int8's which is the size of a byte, as the range of needed numbers is only 7, which can be represented with 3 bits, although you can't really use less than a byte with almost all programming languages because of how memory works. This is recommended even if you have massive memory, as your program will be significantly faster with more efficient caching. **DO NOT USE THIS FOR THE FEATURE SELECTION LANDSCAPES, THEY ARE SUPPOSED TO BE DOUBLES.**

This can be done in all the languages like: Julia, C++, Rust easily. It can also be done in Python, however you need to use the Numpy library. Example with numpy initialization: `fitnessTriangle = numpy.zeros(sizeOfArray, dtype=numpy.int8)`. Then filling the function normally.

If you are still struggling with memory after this, close other programs and remove your algorithm storing a bunch of extra information. Your algorithm behaviour tracking for example may run out of memory if you are storing the entire landscape. You are not required to visualize the algorithm behaviour for this part, although it may help you understand why your algorithm fails. We are releasing this a day early precisely to deal with issues like this.

Generating the Landscape

Another hint is generating the fitness landscape, because of the extremely high size, your construction algorithm is possibly gonna struggle. If the Complexity of your algorithm is too high it might take ages to construct. It will take some time regardless, especially in Python, but here

is some example code to speed it up hopefully a little.

Firstly, just loop over all the positions in the landscape (it's a good idea to parallelize this), and for every point run this algorithm or something very similar:

```
int countActiveBits(int position){
    int changeablePosition = position;
    int countActive = 0;
    for(int i = 0; i<nSize; i++){
        int shift = 1<<(nSize-1-i); //Left shifts equivalent to 2^(position of bit) but faster,
                                     //your compiler probably does this automatically
        if(shift>changeablePosition){
        }
        else{
            changeablePosition -= shift;
            countActive++;
        }
    }
    return countActive;
}
```

Then you lookup the result of this function in this list (**UPDATED**):

[0, 1, 2, 3, 4,5,4,3,2,1, 0, 1, 2, 3, 4, 5, 4, 3, 2, 1, 0, 1, 2, 3, 4, 5, 4, 3, 2, 1, 0, 6].

And store it in your uint8/int8 lookup table as the fitness for that position. You may need to cast it to uint8.

If you have something faster use that by all means, this is just a quick and easy way of doing it.